

Archivo 4 — Guía de Estudio

Alcance y uso de esta guía

Esta guía extrae los conceptos técnicos empleados en el proyecto zz-js01 y los ordena por **dificultad creciente**, de modo que cada concepto se apoye en los anteriores. El alcance queda limitado al lenguaje JavaScript y al ecosistema indispensable para este proyecto: el protocolo HTTP y el estilo REST, el formato JSON, el framework Express y la base de datos SQLite.

Cada concepto se presenta con tres elementos:

- **Definición:** una explicación breve, pensada para quien no conoce el término.
- **Dónde aparece:** el archivo o los archivos del proyecto donde el concepto se utiliza, para poder verificarlo en el código real.
- **Prerrequisito:** el aviso "se recomienda dominar antes..." que indica qué conceptos previos conviene tener firmes.

Se recomienda estudiar los niveles en orden: los conceptos de cada nivel utilizan los de los niveles anteriores.

Nivel 1 — Fundamentos del lenguaje

1.1 Variables: `const` y `let`

- **Definición:** una variable es un nombre que guarda un valor para usarlo más adelante. `const` declara una variable cuyo nombre no puede reasignarse después de creada; `let` declara una variable cuyo valor sí puede cambiar durante la ejecución.
- **Dónde aparece:** `const` se utiliza en todos los archivos del proyecto (por ejemplo, las referencias al DOM en `public/index.html` y la app de Express en `src/app.js`); `let` guarda el estado que cambia, como `idEnEdicion` y `productosActuales` en `public/index.html`.
- **Prerrequisito:** ninguno; es el punto de partida del recorrido.

1.2 Tipos de datos primitivos

- **Definición:** los tipos primitivos son las clases de valores básicas del lenguaje: `string` (texto), `number` (número, con o sin decimales), `boolean` (verdadero o falso), `undefined` (valor ausente) y `null` (valor vacío intencional). El operador `typeof` informa el tipo de un valor.
- **Dónde aparece:** `src/validators/productos.js` verifica `typeof datos.nombre !== "string"` antes de medir el largo del nombre; `Number.isFinite` distingue números válidos de otros valores en el precio.
- **Prerrequisito:** se recomienda dominar antes las variables (1.1).

1.3 Funciones: declaración y funciones flecha

- **Definición:** una función es un bloque de código con nombre que se ejecuta cuando se la llama y puede recibir valores (parámetros) y devolver un resultado. Además de la forma clásica (`function nombre() {}`), JavaScript ofrece las **funciones flecha** (`(p) => p.id === id`), escritas en forma más compacta y habituales como argumento de otras funciones.
- **Dónde aparece:** `validarProducto` en `src/validators/productos.js` es una función por declaración; las funciones flecha aparecen en los manejadores de eventos de `public/index.html` (por ejemplo, `(p) => p.id === id`), en `src/routes/productos.js` (los manejadores de cada ruta) y en las pruebas de `test/api.test.js`.
- **Prerrequisito:** se recomienda dominar antes las variables (1.1) y los tipos de datos (1.2).

1.4 Objetos y arrays

- **Definición:** un objeto es una colección de pares clave-valor, como `{ nombre: "Silla", precio: 10 }`; se accede a sus propiedades con punto (`producto.nombre`). Un array es una lista ordenada de valores, como `["a", "b", "c"]`; se accede a sus elementos por posición y se recorre con `for...of`.
- **Dónde aparece:** los productos del proyecto son objetos con `id`, `nombre` y `precio`; la lista que devuelve `GET /api/productos` es un array de objetos. El array `errores` de `src/validators/productos.js` se completa con `push`.
- **Prerrequisito:** se recomienda dominar antes las variables (1.1) y los tipos de datos (1.2).

1.5 Plantillas de texto (template literals)

- **Definición:** un template literal es un texto delimitado por comillas invertidas (```) que admite interpolar valores con `${expresión}`: el resultado es un único string con los valores ya insertados.
- **Dónde aparece:** `public/index.html` arma las URLs ``/api/productos/${id}`` para PUT y DELETE; `src/server.js` muestra el mensaje ``Servidor corriendo en http://localhost:${PORT}``.
- **Prerrequisito:** se recomienda dominar antes las variables (1.1) y el tipo `string` (1.2).

1.6 Desestructuración

- **Definición:** la desestructuración es una sintaxis que extrae propiedades de un objeto — o elementos de un array — directamente en variables, con la forma `const { a, b } = objeto`.
- **Dónde aparece:** `src/routes/productos.js` extrae la conexión con `const { db } = require("../db");`; `src/app.js` hace lo propio con el router; `test/poblar.test.js` extrae la columna contada con `const { c } = db.prepare("SELECT COUNT(*) AS c FROM productos").get();`.
- **Prerrequisito:** se recomienda dominar antes los objetos y arrays (1.4).

1.7 Método find()

- **Definición:** `find()` es un método de los arrays que recibe una función y devuelve el **primer** elemento que cumple la condición, o `undefined` si ninguno la cumple.
- **Dónde aparece:** `public/index.html`, en `editarProducto(id)`, busca el producto elegido dentro de `productosActuales` con `productosActuales.find((p) => p.id === id)`; la prueba de listado de `test/api.test.js` usa el mismo método para verificar que el producto creado figura en la lista.
- **Prerrequisito:** se recomienda dominar antes los arrays (1.4) y las funciones flecha (1.3).

Nivel 2 — El navegador y el DOM

2.1 Obtención de referencias: `getElementById`

- **Definición:** el DOM (Document Object Model) es la representación de la página como árbol de objetos que JavaScript puede leer y modificar. `document.getElementById("id")` devuelve el elemento del árbol cuyo atributo `id` coincide con el texto indicado.
- **Dónde aparece:** `public/index.html` guarda en constantes, al inicio del script, las referencias al formulario, al cuerpo de la tabla y a los dos botones fijos.
- **Prerrequisito:** se recomienda dominar antes todo el Nivel 1 y nociones básicas de HTML (etiquetas y atributos).

2.2 Eventos y `addEventListener` (submit, click)

- **Definición:** un evento es un suceso del navegador — un clic, un envío de formulario — al que se puede reaccionar. `elemento.addEventListener("evento", función)` registra la función que se ejecutará cuando el evento ocurra. En el envío de formularios, `evento.preventDefault()` cancela el comportamiento por defecto (recargar la página).
- **Dónde aparece:** `public/index.html` registra el manejador `submit` del formulario, el `click` del botón Cancelar y los `click` de los botones Editar y Borrar de cada fila.
- **Prerrequisito:** se recomienda dominar antes `getElementById` (2.1) y las funciones flecha (1.3).

2.3 Creación de nodos: `createElement`, `textContent` y `appendChild`

- **Definición:** `document.createElement("tag")` crea un elemento nuevo vacío; `elemento.textContent = valor` inserta texto plano (sin interpretar HTML); `padre.appendChild(hijo)` agrega el hijo al final del contenido del padre. La combinación de los tres permite construir la página desde JavaScript.
- **Dónde aparece:** `public/index.html`, en `cargarProductos()`: cada fila de la tabla se arma con `createElement("tr")` y `createElement("td")`, se llena con `textContent` y se cuelga con `appendChild`.
- **Prerrequisito:** se recomienda dominar antes `getElementById` (2.1) y los eventos (2.2).

2.4 Formularios: `required`, `type`, `number`, `parseFloat` y `reset`

- **Definición:** los formularios HTML ofrecen validaciones del lado del navegador: el atributo `required` impide enviar un campo vacío y `type="number"` restringe el contenido a números. El valor leído con `.value` es siempre texto, por lo que `parseFloat()` lo convierte en número. `formulario.reset()` limpia todos los campos.
- **Dónde aparece:** `public/index.html` declara los campos con `required` y `step="0.01"`; el manejador del submit aplica `parseFloat` al precio; `salirDeEdicion()` llama a `formulario.reset()`.
- **Prerrequisito:** se recomienda dominar antes las referencias al DOM (2.1), los eventos (2.2) y los tipos de datos (1.2).

2.5 Clausuras en manejadores

- **Definición:** una clausura (closure) es una función que recuerda las variables de su entorno de creación, aunque ese entorno ya haya terminado de ejecutarse. Permite que cada manejador guarde un valor propio.
- **Dónde aparece:** `public/index.html`: cada botón Borrar de la tabla se crea con `botonBorrar.addEventListener("click", () => borrarProducto(producto.id))`; la función flecha "recuerda" el `producto.id` de esa fila en particular.
- **Prerrequisito:** se recomienda dominar antes las funciones flecha (1.3), los eventos (2.2) y la creación de nodos (2.3).

Nivel 3 — Asincronismo y HTTP

3.1 El modelo petición/respuesta

- **Definición:** HTTP (HyperText Transfer Protocol) es el protocolo con el que se comunican navegador y servidor web. Funciona por pares: el cliente envía una **petición** (request) y el servidor devuelve una **respuesta** (response) con un código de estado y, en general, un contenido.
- **Dónde aparece:** todo el proyecto: el navegador de `public/index.html` envía peticiones a `/api/productos` y el servidor Express de `src/app.js` y `src/routes/productos.js` responde cada una.
- **Prerrequisito:** se recomienda dominar antes los Niveles 1 y 2.

3.2 Verbos GET/POST/PUT/DELETE

- **Definición:** el verbo HTTP indica la intención de la petición: `GET` (obtener datos), `POST` (crear un recurso), `PUT` (reemplazar un recurso existente) y `DELETE` (borrarlo). Este conjunto, junto con URLs que identifican recursos, caracteriza el estilo REST.
- **Dónde aparece:** `src/routes/productos.js` define las cuatro rutas: `router.get("/")`, `router.post("/")`, `router.put("/:id")` y `router.delete("/:id")`; `public/index.html` las consume con `fetch`.
- **Prerrequisito:** se recomienda dominar antes el modelo petición/respuesta (3.1).

3.3 Códigos de estado

- **Definición:** cada respuesta HTTP incluye un código numérico que resume el resultado. La familia 2xx indica éxito (200 OK, 201 Created, 204 No Content) y la familia 4xx indica un error del cliente (400 Bad Request, 404 Not Found).
- **Dónde aparece:** `src/routes/productos.js` responde con 200, 201, 204, 400 y 404 según el caso; `test/api.test.js` verifica cada código; `public/index.html` consulta `respuesta.ok` para distinguir el éxito.
- **Prerrequisito:** se recomienda dominar antes los verbos HTTP (3.2).

3.4 JSON: stringify y parse

- **Definición:** JSON (JavaScript Object Notation) es un formato de texto para intercambiar datos, con la sintaxis de objetos y arrays de JavaScript. `JSON.stringify(objeto)` convierte un objeto en texto JSON (serializar); `JSON.parse(texto)` hace la conversión inversa. `respuesta.json()` de `fetch` parsea el cuerpo de una respuesta.
- **Dónde aparece:** `public/index.html` serializa el producto con `JSON.stringify({ nombre, precio })` y lee las respuestas con `respuesta.json()`; el servidor responde con `res.json(...)`.
- **Prerrequisito:** se recomienda dominar antes los objetos y arrays (1.4) y el tipo `string` (1.2).

3.5 async/await

- **Definición:** una operación asíncrona es la que tarda en completarse (por ejemplo, esperar la red). `async` marca una función que puede usar `await`; `await` pausa esa función hasta que el resultado esté disponible, sin congelar el resto de la página. El resultado pendiente se representa con una **promesa**.
- **Dónde aparece:** `src/server.js` envuelve el arranque en `async function main()`; `public/index.html` declara `async` los manejadores que llaman a `fetch` y encadena `guardarProducto` y `cargarProductos` con `await`.
- **Prerrequisito:** se recomienda dominar antes las funciones (1.3) y el modelo petición/respuesta (3.1).

3.6 fetch con rutas relativas

- **Definición:** `fetch(url, opciones)` es la función del navegador que envía peticiones HTTP. Acepta un objeto de opciones con `method` (verbo), `headers` (encabezados, como el tipo de contenido) y `body` (datos). Una ruta relativa como `"/api/productos"` se completa con el origen de la página (`http://localhost:3000`), por eso el HTML debe servirse por HTTP y no abrirse como archivo local.
- **Dónde aparece:** `public/index.html` realiza las cuatro llamadas: GET en `cargarProductos()`, POST/PUT en `guardarProducto()` y DELETE en `borrarProducto()`.
- **Prerrequisito:** se recomienda dominar antes los verbos (3.2), los códigos de estado (3.3), JSON (3.4) y `async/await` (3.5).

Nivel 4 — El servidor con Node.js y Express

4.1 Node.js y npm

- **Definición:** Node.js es un entorno que ejecuta JavaScript fuera del navegador, sobre todo en servidores. npm es su gestor de paquetes: descarga librerías de terceros declaradas en `package.json` (con `npm install`) y ejecuta los comandos definidos en su sección `scripts`. Este proyecto requiere Node 22 o superior.
- **Dónde aparece:** `package.json` declara las dependencias (`express`, `better-sqlite3`, `supertest`) y los scripts `start` y `test`; todo el directorio `src/` corre sobre Node.
- **Prerrequisito:** se recomienda dominar antes el Nivel 1.

4.2 Módulos CommonJS: `require` y `module.exports`

- **Definición:** CommonJS es el sistema de módulos de Node: cada archivo es un módulo. `require("./archivo")` importa lo que otro archivo exportó, y `module.exports` define qué exporta el archivo actual. Node guarda en caché los módulos ya cargados: un mismo `require` devuelve siempre el mismo objeto.
- **Dónde aparece:** todos los archivos de `src/` y `test/`; por ejemplo, `src/app.js` requiere `./routes/productos` y `src/db.js` exporta `module.exports = { db, poblarDesdeArchivo };`.
- **Prerrequisito:** se recomienda dominar antes Node y npm (4.1), los objetos (1.4) y la desestructuración (1.6).

4.3 Creación de la aplicación

- **Definición:** `const app = express()` crea el objeto aplicación de Express, el framework web que organiza el servidor como una cadena de middlewares y rutas que responden pedidos HTTP.
- **Dónde aparece:** `src/app.js` crea la app, le agrega los middlewares y las rutas, y la exporta con `module.exports = app;`.
- **Prerrequisito:** se recomienda dominar antes Node y npm (4.1) y los módulos CommonJS (4.2).

4.4 Middlewares: concepto y cadena

- **Definición:** un middleware es una función de Express que recibe la petición y la respuesta y puede procesarlas o dejarlas pasar. Los middlewares y rutas se ejecutan **en el orden en que se registran**, de arriba hacia abajo, hasta que alguno responde.
- **Dónde aparece:** `src/app.js` registra, en este orden: `express.json()`, `express.static(...)` y el router de productos. El orden es parte del diseño: el body debe interpretarse antes de que las rutas lo lean.
- **Prerrequisito:** se recomienda dominar antes la creación de la app (4.3) y las funciones (1.3).

4.5 `express.json`

- **Definición:** `express.json()` es el middleware que interpreta el cuerpo JSON de las peticiones POST y PUT y lo deja disponible como objeto JavaScript en `req.body`. Sin él, `req.body` llega `undefined`.
- **Dónde aparece:** `src/app.js`, primera línea de configuración de la app.
- **Prerrequisito:** se recomienda dominar antes los middlewares (4.4) y JSON (3.4).

4.6 express.static

- **Definición:** `express.static(carpeta)` es el middleware que sirve los archivos de una carpeta tal como están: si el navegador pide `/`, Express entrega `index.html` de esa carpeta.
- **Dónde aparece:** `src/app.js` lo apunta a la carpeta `public/` del proyecto; por eso el frontend se abre en `http://localhost:3000/`.
- **Prerrequisito:** se recomienda dominar antes los middlewares (4.4).

4.7 Parámetros de ruta (:id)

- **Definición:** un parámetro de ruta es un segmento variable de la URL, escrito con dos puntos (`/:id`). Express lo entrega en `req.params.id`. Llega siempre como texto (string), aunque represente un número.
- **Dónde aparece:** `src/routes/productos.js` define `/:id` en PUT y DELETE; ambas rutas lo pasan a las consultas SQL y PUT lo convierte con `Number(req.params.id)` para la respuesta.
- **Prerrequisito:** se recomienda dominar antes la creación de la app (4.3) y los verbos HTTP (3.2).

4.8 Router

- **Definición:** `express.Router()` crea una mini-aplicación que agrupa rutas relacionadas. La app principal la monta con `app.use(prefijo, router)`: las rutas internas se sirven bajo ese prefijo.
- **Dónde aparece:** `src/routes/productos.js` crea el router con las rutas `/` y `/:id`; `src/app.js` lo monta con `app.use("/api/productos", productosRouter)`, de modo que `/` interno responde en `/api/productos` y `/:id` en `/api/productos/:id`.
- **Prerrequisito:** se recomienda dominar antes la creación de la app (4.3), los middlewares (4.4) y los parámetros de ruta (4.7).

Nivel 5 — Persistencia de datos

5.1 SQLite como base embebida

- **Definición:** una base de datos es un sistema que almacena datos organizados en tablas (filas y columnas). SQLite es una base **embebida**: no necesita un servidor aparte, porque toda la base vive en un único archivo del disco que el programa abre directamente.
- **Dónde aparece:** `src/db.js` abre o crea `mi_base_de_datos.db` (salvo que la variable `DB_PATH` indique otra ruta); el archivo aparece en la raíz del proyecto.
- **Prerrequisito:** se recomienda dominar antes Node y npm (4.1).

5.2 better-sqlite3 y su naturaleza síncrona

- **Definición:** un driver es la librería que conecta el programa con la base de datos. `better-sqlite3` es el driver de este proyecto y es **síncrono**: cada consulta se ejecuta y devuelve su resultado de inmediato, sin promesas ni callbacks, bloqueando el proceso hasta terminar.
- **Dónde aparece:** `src/db.js` crea la conexión con `new Database(DB_PATH)`; `src/routes/productos.js` consulta con `db.prepare(...).all()` y `.run()`.
- **Prerrequisito:** se recomienda dominar antes SQLite como base embebida (5.1) y los módulos CommonJS (4.2).

5.3 SQL básico

- **Definición:** SQL es el lenguaje de consulta de las bases relacionales. Las cinco sentencias usadas en el proyecto son: `CREATE TABLE IF NOT EXISTS` (crear la tabla si falta), `SELECT` (leer filas), `INSERT` (insertar), `UPDATE` (modificar, con `WHERE` para limitar qué filas) y `DELETE` (borrar, también con `WHERE`). Omitir el `WHERE` afecta a **todas** las filas.
- **Dónde aparece:** la tabla se crea en `src/db.js`; las sentencias de datos viven en `src/routes/productos.js`; `poblar_base.sql` contiene los `INSERT` de los 5 productos de ejemplo.
- **Prerrequisito:** se recomienda dominar antes better-sqlite3 (5.2).

5.4 Prepared statements y prevención de inyección SQL

- **Definición:** un prepared statement es una consulta que se compila una vez con marcadores `?` en lugar de valores; los valores se envían después, por separado, con `run(...)`. Como los datos nunca se pegan dentro del texto SQL, resulta imposible que un dato del usuario altere la consulta: esa es la prevención de la inyección SQL. Además, la consulta compilada se reutiliza con eficiencia.
- **Dónde aparece:** las cuatro consultas de `src/routes/productos.js` (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) usan `prepare` con `?`; también la verificación del seed en `test/poblar.test.js`.
- **Prerrequisito:** se recomienda dominar antes el SQL básico (5.3) y los tipos de datos (1.2).

Nivel 6 — Integración y calidad

6.1 Validación del lado del servidor (y su diferencia con la del cliente)

- **Definición:** validar es verificar que los datos cumplan las reglas antes de usarlos. La validación del **cliente** (atributos `required` del formulario) ayuda al usuario, pero puede saltarse con herramientas como `curl`; la validación del **servidor** es la única que garantiza las reglas. El proyecto acumula **todos** los errores encontrados y los devuelve juntos.
- **Dónde aparece:** `src/validators/productos.js` define `validarProducto` (nombre: texto obligatorio de hasta 100 caracteres; precio: número finito, mínimo 0.01); las rutas POST y PUT la invocan antes de tocar la base y responden 400 con la lista de errores.

- **Prerrequisito:** se recomienda dominar antes los objetos (1.4), los códigos de estado (3.3) y las rutas de Express (4.7, 4.8).

6.2 Semántica de códigos de estado (201/204/400/404)

- **Definición:** más allá de la familia, cada código tiene un significado preciso y conviene responder con el correcto: 201 para "se creó un recurso", 204 para "éxito sin contenido que devolver", 400 para "datos inválidos" y 404 para "recurso inexistente". En el proyecto, un `changes === 0` en UPDATE o DELETE (ninguna fila afectada) se traduce en un 404 honesto en lugar de un éxito silencioso.
- **Dónde aparece:** `src/routes/productos.js` en las cuatro rutas; `test/api.test.js` verifica cada caso, incluidos PUT y DELETE sobre ids inexistentes.
- **Prerrequisito:** se recomienda dominar antes los códigos de estado (3.3) y el SQL básico (5.3).

6.3 Variables de entorno

- **Definición:** una variable de entorno es un valor que el sistema operativo pasa al proceso al arrancarlo, sin escribirlo en el código. En Node se leen con `process.env.NOMBRE`. Permiten cambiar el comportamiento — una ruta de archivo, un puerto — sin editar el programa.
- **Dónde aparece:** `DB_PATH` elige el archivo de la base en `src/db.js` y `src/server.js`; `PORT` elige el puerto en `src/server.js`; ambos las leen con el patrón `process.env.DB_PATH || "valor por defecto"`.
- **Prerrequisito:** se recomienda dominar antes Node y npm (4.1).

6.4 Estructura modular de un proyecto

- **Definición:** organizar un proyecto en módulos consiste en repartir responsabilidades entre archivos con un propósito único cada uno. Facilita leerlo, probarlo y modificarlo sin efectos secundarios.
- **Dónde aparece:** `src/` separa la aplicación (`app.js`), el arranque (`server.js`), la conexión (`db.js`), las rutas (`routes/productos.js`) y las reglas de validación (`validators/productos.js`); el frontend vive aislado en `public/` y las pruebas en `test/`.
- **Prerrequisito:** se recomienda dominar antes los módulos CommonJS (4.2) y el router (4.8).

6.5 Pruebas automatizadas

- **Definición:** una prueba automatizada es un programa que ejercita el código y verifica resultados esperados sin intervención humana. `node:test` es el corredor de pruebas incluido en Node; `supertest` dispara peticiones HTTP contra la app Express sin levantar un servidor real. El **aislamiento** se logra apuntando `DB_PATH`, antes de cualquier `require` de `src/`, a un archivo temporal único: las pruebas nunca tocan la base real.
- **Dónde aparece:** `test/api.test.js` (12 pruebas: la cadena feliz del CRUD y las validaciones) y `test/poblar.test.js` (1 prueba del seed); `npm test` ejecuta ambas con `node --test`.
- **Prerrequisito:** se recomienda dominar antes los módulos CommonJS (4.2), las variables de entorno (6.3), el CRUD del servidor (Niveles 4 y 5) y la validación (6.1).

Cierre

El recorrido recomendado es, en síntesis, el propio orden de esta guía: primero el lenguaje (Nivel 1), porque todo lo demás se escribe con él; después el navegador y su DOM (Nivel 2), que dan sentido al frontend; luego el asincronismo y HTTP (Nivel 3), que conectan las dos mitades del proyecto; a continuación Node.js y Express (Nivel 4), que conforman el servidor; después la persistencia con SQLite (Nivel 5), que guarda los datos; y, por último, la integración y la calidad (Nivel 6), que unen todas las piezas con validación, códigos de estado bien elegidos, configuración externa, estructura modular y pruebas automatizadas. Estudiado en ese orden, cada concepto nuevo encuentra apoyados todos los que requiere, y el proyecto `zz-js01` queda como caso completo donde cada idea tiene un lugar verificable en el código.